SKILL Week 3 Task 1:

To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

→Creating a main loop

First check if src and all other file are available

```
suraj-ubuntu@suraj-ubuntu    :~$ cd ~/OSSP
suraj-ubuntu@suraj-ubuntu    :~/OSSP$ ls
LICENSE   README.md  bin   fork_exec    fork_exec.c.save  obj      src
Makefile  assets     docs  fork_exec.c  include           scripts  tests
suraj-ubuntu@suraj-ubuntu    :~/OSSP$ ls src
builtin.c  executor.c  main.c  parser.c  shell.c
```

So here I added a main loop in main.c in src

```
 GNU nano 7.2                        src/main.c
#include <stdio.h>
#include <string.h>

int main()
{
    while (1)
    {
        char input[1024];

        printf("ossp> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
            printf("\nExiting shell...\n");
            break;
        }

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }

        printf("You entered: %s\n", input);
    }

    return 0;
}
```

Now compile it

```
suraj-ubuntu@suraj-ubuntu:~/OSSP$ gcc -o fork_exec fork_exec.c
suraj-ubuntu@suraj-ubuntu:~/OSSP$ ./fork_exec
Hello from parent!
Hello from child!
suraj-ubuntu@suraj-ubuntu:~/OSSP$ cd bin
suraj-ubuntu@suraj-ubuntu:~/OSSP/bin$ ls
fork_exec
fork_exec.c
fork_exec.c.save
Makefile
```

CONTROL FLOW

OSSP/

├── src/

│ └── main.c ← Task 1 main loop

│

├── bin/

│ └── shell ← compiled program

│

├── include/

├── obj/

├── docs/

├── tests/

├── scripts/

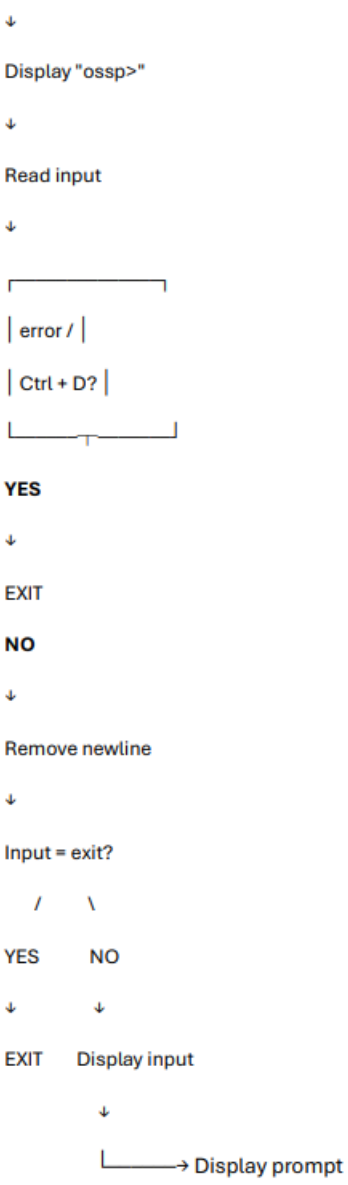
├── assets/

├── Makefile

├── README.md

└── LICENSE

START



This part indicates testing of interactive loops

```
suraj-ubuntu@suraj-ubuntu::~/OSSP$ ./bin/shell
ossp> hello
You entered: hello
ossp> test
You entered: test
ossp> exit
Exiting shell...
```

Task 2

1) Open()

```
suraj-ubuntu@suraj-ubuntu :~$ cd ~/OSSP
suraj-ubuntu@suraj-ubuntu :~/OSSP$ pwd
/home/suraj/OSSP
suraj-ubuntu@suraj-ubuntu :~/OSSP$ touch File1.txt
suraj-ubuntu@suraj-ubuntu :~/OSSP$ ls
File1.txt   README.md  docs         fork_exec.c.save  scripts
LICENSE     assets     fork_exec    include           src
Makefile    bin        fork_exec.c  obj               tests
suraj-ubuntu@suraj-ubuntu :~/OSSP$ nano File1.txt
```

After creating file one

```
  GNU nano 7.2                        File1.txt
This is file1
This content will be copied to File2.
```

Create copy.c

```
suraj-ubuntu@suraj-ubuntu : ~/OSSP$ cat File1.txt
This is file1
This content will be copied to File2.
suraj-ubuntu@suraj-ubuntu:: ~/OSSP$ nano copy.c
```

```
  GNU nano 7.2                          copy.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int f1, f2;

    f1 = open("File1.txt", O_RDONLY);
    f2 = open("File2.txt", O_WRONLY | O_CREAT, 0644);

    printf("Files opened successfully\n");

    close(f1);
    close(f2);

    return 0;
}
```

```
suraj-ubuntu@suraj-ubuntu: ~/OSSP$ gcc copy.c -o copy
suraj-ubuntu@suraj-ubuntu: ~/OSSP$ ls
File1.txt    README.md   copy     fork_exec          include  src
LICENSE      assets      copy.c   fork_exec.c        obj      tests
Makefile     bin         docs     fork_exec.c.save   scripts
suraj-ubuntu@suraj-ubuntu:~/OSSP$ ./copy
Files opened successfully
suraj-ubuntu@suraj-ubuntu: ~/OSSP$ ls
File1.txt    Makefile    bin      docs          fork_exec.c.save  scripts
File2.txt    README.md   copy     fork_exec     include           src
LICENSE      assets      copy.c   fork_exec.c   obj               tests
```

The files are opened successfully!

2. Read()

Now to read the file I have slightly edited copy.c

```
 GNU nano 7.2                          copy.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int f1, f2;
    char buffer[100];

    f1 = open("File1.txt", O_RDONLY);
    f2 = open("File2.txt", O_WRONLY | O_CREAT, 0644);

    read(f1, buffer, 100);

    printf("%s", buffer);

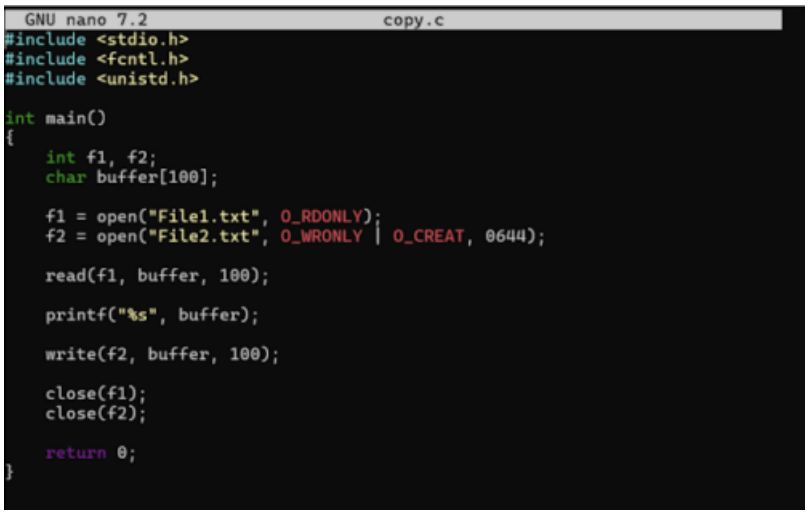
    close(f1);
    close(f2);
    return 0;
}
```

```
suraj-ubuntu@suraj-ubuntu: ~/OSSP$ gcc copy.c -o copy
suraj-ubuntu@suraj-ubuntu: ~/OSSP$ ./copy
This is file1
This content will be copied to File2.
```

So after editing copy.c giving the read function we have successfully read file1.

3. Write()

Again open copy.c add write element



```
 GNU nano 7.2                          copy.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

int main()
{
    int f1, f2;
    char buffer[100];

    f1 = open("File1.txt", O_RDONLY);
    f2 = open("File2.txt", O_WRONLY | O_CREAT, 0644);

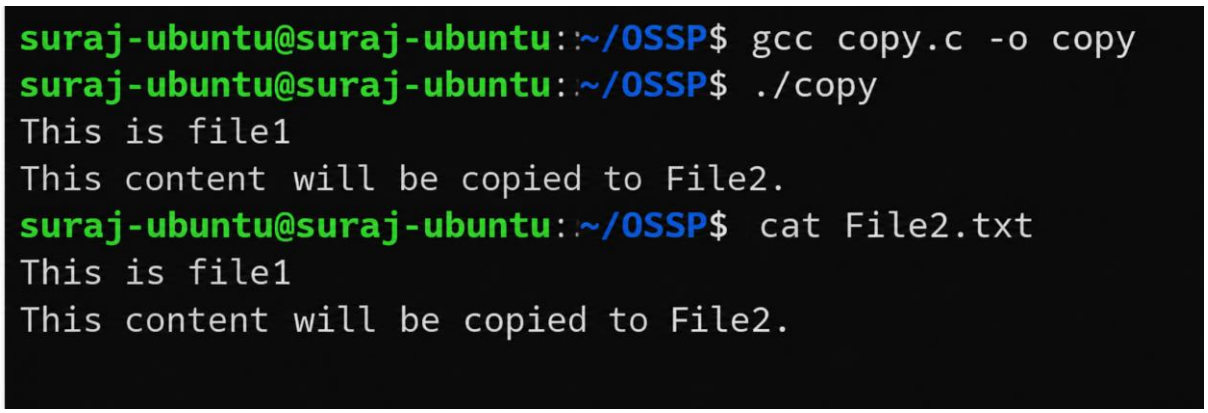
    read(f1, buffer, 100);

    printf("%s", buffer);

    write(f2, buffer, 100);

    close(f1);
    close(f2);

    return 0;
}
```



```
suraj-ubuntu@suraj-ubuntu:~/OSSP$ gcc copy.c -o copy
suraj-ubuntu@suraj-ubuntu:~/OSSP$ ./copy
This is file1
This content will be copied to File2.
suraj-ubuntu@suraj-ubuntu:~/OSSP$ cat File2.txt
This is file1
This content will be copied to File2.
```

After copying the text its time to close it

Task 3:

Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction.

```
samanth-ubuntu@samanth-ubuntu   :~$ cd ~/OSSP
samanth-ubuntu@samanth-ubuntu   :~/OSSP$ cd src
samanth-ubuntu@samanth-ubuntu   :~/OSSP/src$ nano input.c
samanth-ubuntu@samanth-ubuntu   :~/OSSP/src$ gcc input.c -o input
samanth-ubuntu@samanth-ubuntu   :~/OSSP/src$ ./input
Enter a character: K

You entered: K
samanth-ubuntu@samanth-ubuntu   :~/OSSP/src$ nano input.c
samanth-ubuntu@samanth-ubuntu   :~/OSSP/src$ gcc input.c -o input
samanth-ubuntu@samanth-ubuntu   :~/OSSP/src$ ./input
shell> hello

Command entered: hello
samanth-ubuntu@samanth-ubuntu   :~/OSSP/src$ ./input
shell> ls

Command entered: ls
samanth-ubuntu@samanth-ubuntu   :~/OSSP/src$ ./input
shell> pwd

Command entered: pwd
samanth-ubuntu@samanth-ubuntu   :~/OSSP/src$ nano input.c
samanth-ubuntu@samanth-ubuntu   :~/OSSP/src$ gcc input.c -o input
samanth-ubuntu@samanth-ubuntu   :~/OSSP/src$ ./input
shell> hello
hello
Command entered: hello
shell> ls
ls
Command entered: ls
shell> pwd
pwd
Command entered: pwd
shell> exit
exit
Command entered: exit
Exiting shell...
samanth-ubuntu@samanth-ubuntu   :~/OSSP/src$
```

```
            /* DOWN */
            else if (ch == 'B')
            {
                if (h < count - 1)
                {
                    h++;
                    printf("\rShell> %s", history[h]);
                    strcpy(input, history[h]);
                    pos = strlen(input);
                }
            }
        }

        /* Normal character */
        else
        {
            input[pos] = ch;
            pos++;
            printf("%c", ch);
        }

        fflush(stdout);
    }

    if (strcmp(input, "exit") == 0)
        break;
}

tcsetattr(0, TCSANOW, &old);

return 0;
}
```

Now press ↑.

You should see the previous command.

```
suraj-ubuntu@suraj-ubuntu   :~$ cd ~/OSSSP/src
suraj-ubuntu@suraj-ubuntu   :~$ nano history.c
suraj-ubuntu@suraj-ubuntu   :~/OSSP/src$ gcc history.c -o history
suraj-ubuntu@suraj-ubuntu   :~/OSSP/src$ ./history

Shell> hello
Shell> ls
Shell> pwd
Shell> hello
```

```c
    if (head == NULL) {
        free(buffer);
        return 1;
    }

    head->data = 100;
    head->next = NULL;

    struct Node *second = malloc(sizeof(struct Node));

    if (second == NULL) {
        free(head);
        free(buffer);
        return 1;
    }

    second->data = 200;
    second->next = NULL;

    head->next = second;

    printf("\nLinked List:\n");

    struct Node *current = head;

    while (current != NULL) {
        printf("%d -> ", current->data);
        current = current->next;
    }

    printf("NULL\n");

    /* 4. Release memory */
    free(second);
    free(head);
    free(buffer);

    printf("\nMemory released successfully.\n");

    return 0;
}
```

Valgrind Installation

```
suraj-ubuntu@suraj-ubuntu: ~/OSSP/src$ sudo apt update
sudo apt install valgrind
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
82 packages can be upgraded. Run 'apt list --upgradable' to see them.
E: dpkg was interrupted, you must manually run 'sudo dpkg --configure -a' to
correct the problem.
suraj-ubuntu@suraj-ubuntu: ~/OSSP/src$ sudo dpkg --configure -a
Setting up allocate (1.1.9-2ubuntu2) ..
Created symlink /etc/systemd/system/timers.target.wants/allocate-updatedb.tim
er → /usr/lib/systemd/system/allocate-updatedb.timer.
Processing triggers for man-db (2.12.0-4build2) ....
Processing triggers for libc-bin (2.39-0ubuntu8.8)  ...
suraj-ubuntu@suraj-ubuntu: ~/OSSP/src$ valgrind --leak-check=full ./memory
Command 'valgrind' not found, but can be installed with:
sudo snap install valgrind    version 3.20.0, or
sudo apt  install valgrind    version 1:3.22.0-0ubuntu2
See 'snap info valgrind' for additional versions.
suraj-ubuntu@suraj-ubuntu: ~/OSSP/src$ sudo apt install valgrind
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  gdb libbabeltrace1 libcd6-0 libdebuginfod-common libdebuginfod1t64
  libipt2 libsource-highlight-common libsource-highlightt64
Suggested packages:
  gdb-doc gdbserver valgrind-dbg valgrind-mpi kcachegrind alleyoop
  valkyrie
```

```
suraj-ubuntu@suraj-ubuntu: ~/OSSP/src$ valgrind --version
valgrind-3.22.0
suraj-ubuntu@suraj-ubuntu: ~/OSSP/src$ valgrind --leak-check=full ./memory
==1662== Memcheck, a memory error detector
==1662== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==1662== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==1662== Command: ./memory
Enter a string (maximum 4 characters): abc
Buffer contains: abc

Enter another string (maximum 19 characters): hello world
Resized buffer contains: hello world

Linked list:
100 -> 200 -> NULL

Memory released successfully.
==1662==
==1662== HEAP SUMMARY:
==1662==      in use at exit: 0 bytes in 0 blocks
==1662==    total heap usage: 6 allocs, 6 frees, 2,105 bytes allocated
==1662==
==1662== All heap blocks were freed -- no leaks are possible
==1662==
==1662== For lists of detected and suppressed errors, rerun with: -s
==1662== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
suraj-ubuntu@suraj-ubuntu: ~/OSSP/src$
```